Estimating the Size and the Number of Paths in a DAG by Following a Random Path

Günter Rote
Institut für Informatik, Freie Universität Berlin

Abstract

We describe a simple method to compute an unbiased estimate of the number of nodes of a directed acyclic graph (DAG) by following a random directed path. It generalizes in a straightforward way a method of D. E. Knuth from the 1960s for estimating the size of a rooted tree. A slight variation allows to estimate the number of source-sink paths.

We extend the algorithm to use many "agents" that walk cooperatively through the graph. This extension unifies and generalizes several methods that have been proposed for approximately counting the nodes of a tree.

2012 ACM Subject Classification Mathematics of computing → Graph algorithms

Keywords and phrases Approximate counting, random walk, importance sampling, polytopes

Contents

1 Problem statement and the Path Sampling algorithm (Algorithm P)

We are given a directed acyclic graph (DAG) $G = (V, E)$ and a *cost function* $c\colon V \to \mathbb{R}$ on the nodes. The task is to estimate the total cost $c(G) := \sum_{v \in V} c(v)$.

The intended setting of the problem is that G is too large to be fully scanned, but we have access to the neighbors of a node and can thereby explore parts of G. In particular, for the basic Path Sampling algorithm, we make the following assumptions:

- There is a single *source* node s_0 from which all other nodes can be reached by directed paths.
- For any node, we can pick a *successor* node (i. e., a neighbor via an outgoing arc) uniformly at random.
- For any node v, we can determine its outdegree $d^+(v)$, i. e., the number of its *successors*, its indegree $d^-(v)$, i. e., the number of its *predecessors*, and we can access its cost $c(v)$.

The following algorithm computes an unbiased estimate X of $c(G)$ by walking along a directed path from the source. Figure 1 illustrates the algorithm on a small example.

Algorithm P: Path Sampling

```
    // Initialization:
    X := 0   // accumulated estimate
    u := s_0   // current node, starts at the source
    W := 1   // weight
    loop
        // visit u:
        X := X + W · c(u)   // augment the estimate
        if u has no successor, terminate and return X as the estimate
        // otherwise, proceed to a random successor v:
(R)     pick a node v among the d⁺(u) successors of u uniformly at random
(U)     W := W × d⁺(u) / d⁻(v)   // update the weight
        u := v   // go to v
```

Path sampling for trees. Algorithm P was originally developed for rooted trees by Don Knuth in 1962. It was first mentioned in a paper of Hall and Knuth [14] in 1965, and it has been investigated in its own right in [16], see also [17, Sect. 7.2.2, pp. 46–51, Corollary E]. For a tree, the indegree $d^-(v)$ is always 1, and the weight W is just the product of the outdegrees of the visited nodes.

Our algorithm is a straightforward extension to DAGs. To compensate for the fact that there can be several ways to reach a given node v, we divide W by the indegree $d^-(v)$ when going to v.

Node costs. The original purpose and the most wide-spread application of Algorithm P is to estimate the cost of backtracking or branch-and-bound algorithms. The backtracking tree or branch-and-bound tree that such an algorithm traverses may be enormous, while at other times, it is of moderate size and the algorithm terminates resonably fast. If such a program has been running for a while, it would be useful to know whether it is just about to finish or it would continue running for years. Therefore it is desirable to have a good prediction of the remaining running time. In these applications the cost $c(v)$ is the time to process node v.

In other applications, the nodes represent some combinatorial objects, and one wishes to determine the number of these objects. In these situations, one uses uniform costs $c(v) = 1$

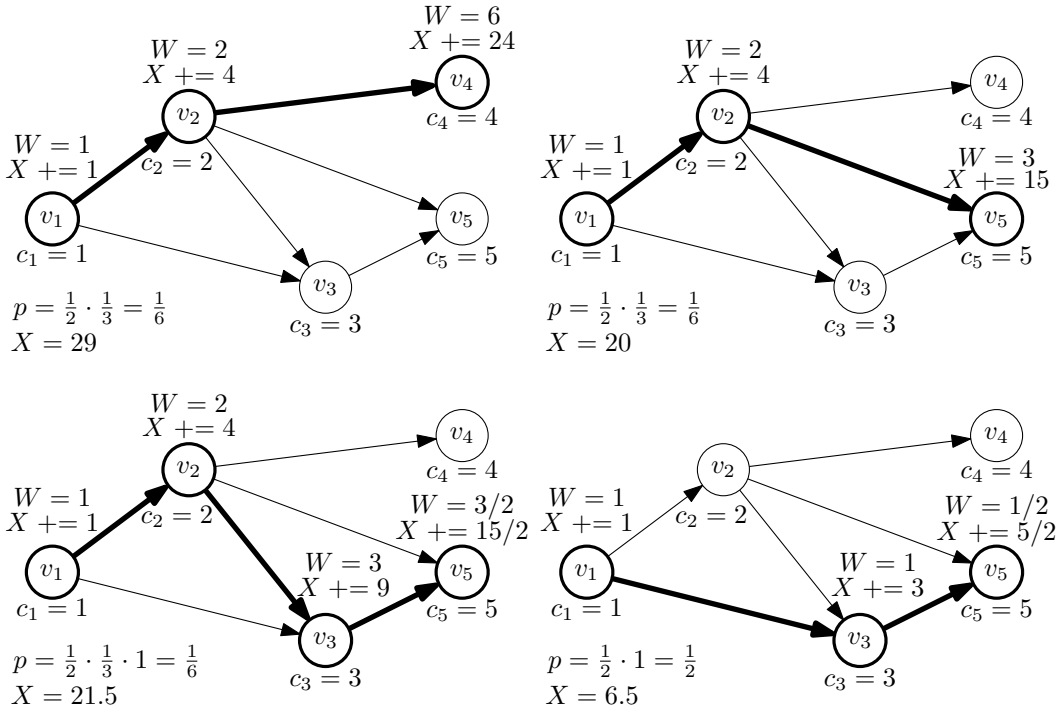

Figure 1 A DAG G with five nodes $v_1, \ldots, v_5$. The costs are $c_i = c(v_i) = i$, for a total cost of $c(G) = 1+2+3+4+5 = 15$. There are four sample paths that Algorithm P can take. The expected estimate is $E[X] = \frac{1}{6} \times 29 + \frac{1}{6} \times 20 + \frac{1}{6} \times 21.5 + \frac{1}{2} \times 6.5 = \frac{29+20+21.5+19.5}{6} = \frac{90}{6} = 15$.

for all nodes v. Avis and Devroye used the tree version of Algorithm P to estimate the number of vertices of a polytope [3]. If one wishes to count only the nodes with certain characteristics, this can be arranged by choosing $c(v) \in \{0, 1\}$.

1.1 Weighted selection

In line (R), the random successor does not have to be selected uniformly, but we can choose arbitrary positive transition probabilities $\vec{p}_{uv}$ subject to the constraint that they form a probability distribution:

$$\sum_v \vec{p}_{uv} = 1, \text{ for all } u \text{ with } d^+(u) > 0 \tag{1}$$

We call the quantities $\vec{p}_{uv}$ the *forward proportions*. (The arrow above the p does not signify a vector, but it reminds us of the direction in which $\vec{p}_{uv}$ operates.)

Similarly, the incoming edges at a node v need not be weighted equally, but they can be weighted by quantities $\overleftarrow{q}_{uv}$ that sum to 1 for the incoming arcs of each node v:

$$\sum_u \overleftarrow{q}_{uv} = 1, \text{ for all } v \neq s_0 \tag{2}$$

We call the quantities $\overleftarrow{q}_{uv}$ the *backward proportions*. (They don't have a probabilistic interpretation, and they can even be negative.)

The update of the weight when traversing the arc uv in line (U) is replaced by

$$(\mathrm{U}_q) \qquad W := W \times \overleftarrow{q}_{uv} \,/\, \vec{p}_{uv}.$$

The standard choices are $\vec{p}_{uv} = 1/d^+(u)$ and $\overleftarrow{q}_{uv} = 1/d^-(v)$.

▶ **Theorem 1.** *For arbitrary positive forward proportions $\vec{p}_{uv}$ satisfying* (1) *and backward proportions $\overleftarrow{q}_{uv}$ satisfying* (2)*, Algorithm P produces an unbiased estimate of $c(G)$, i. e.,*

$$E[X] = c(G).$$

We will prove this below as a consequence of a more general statement.

1.2 No backward proportions: counting paths

If we omit the backward proportions, we get an estimate of the *number of paths starting from the source*. The weight updated step (U) is replaced by

$$(\mathrm{U}_1) \qquad W := W \,/\, \vec{p}_{uv}.$$

▶ **Theorem 2.** *For arbitrary positive forward proportions $\vec{p}_{uv}$ satisfying* (1)*, Algorithm P with update step* (U_1) *produces an estimate X whose expected value is*

$$E[X] = \sum_{v \in V} c(v) \cdot \#(\textit{paths from } s_0 \textit{ to } v \textit{ in } G).$$

As a special case, we can set $c(v) = 1$ if v is a sink and $c(v) = 0$ otherwise, and we get the number of source-sink paths.

More generally, we can replace the backward proportions by arbitrary real arc weights $e_{uv} \in \mathbb{R}$. The weight function is extended from arcs to paths P in the usual way, by setting $e(P)$ to be the product of the arc weights along P:

$$e(u_0 u_1 \dots u_k) = e_{u_0 u_1} e_{u_1 u_2} \dots e_{u_{k-1} u_k}$$

▶ **Theorem 3.** *For arbitrary positive forward proportions $\vec{p}_{uv}$ satisfying* (1) *and arbitrary arc weights e_{uv}, Algorithm P with update step*

$$(\mathrm{U}_e) \qquad W := W \times e_{uv} \,/\, \vec{p}_{uv}$$

produces an estimate X whose expected value is

$$E[X] = c_e(G) := \sum_{v \in V} \sum_{\text{paths } P \text{ from } s_0 \text{ to } v} e(P) c(v). \tag{3}$$

1.3 Analysis of the expectation

Proof of Theorem 3. We define for every node v the random variable

$$W_v := \begin{cases} \text{the value of } W \text{ at the time that } v \text{ was visited}, & \text{if } v \text{ was visited} \\ 0, & \text{if } v \text{ was not visited} \end{cases}$$

Then $X = \sum_{v \in V} W_v \cdot c(v)$, and therefore, $E[X] = \sum_{v \in V} E[W_v] \cdot c(v)$, by linearity of expectation. The theorem is a direct consequence of the following lemma.

▶ **Lemma 4.** *For every node v, $E[W_v] = \sum_{\text{paths } P \text{ from } s_0 \text{ to } v} e(P)$.*

Proof. By definition, the expectation of W_v can be written as follows:

$$E[W_v] = \sum_{\text{paths } P \text{ from } s_0 \text{ to } v} \Pr[\text{path } P \text{ is taken}] \times (W_v \text{ as a result of path } P) \tag{4}$$

For a path $P = u_0u_1 \dots u_k$,

$$\Pr[\text{path } P \text{ is taken}] = \frac{1}{\vec{p}_{u_0u_1}\vec{p}_{u_1u_2} \cdots \vec{p}_{u_{k-1}u_k}}.$$

If the path P is taken, then

$$W_v = \frac{e_{u_0u_1}}{\vec{p}_{u_0u_1}} \cdot \frac{e_{u_1u_2}}{\vec{p}_{u_1u_2}} \cdots \frac{e_{u_{k-1}u_k}}{\vec{p}_{u_{k-1}u_k}}.$$

Thus, in each product term in (4), the $\vec{p}$'s cancel, and the claim of the lemma follows. The proof of Theorem 3 is thus also complete. ◀

Theorem 2 is just a special case of Theorem 3 with uniform arc weights $e \equiv 1$.

Proof of Theorem 1 from Theorem 3. Here the weights are the backward proportions $e_{uv} = \overleftarrow{q}_{uv}$. We only need to show the following statement:

▶ **Lemma 5.** *For every node v,*

$$\sum_{\text{paths } P \text{ from } s_0 \text{ to } v} e(P) = 1.$$

Proof. This is easy to prove by induction on the length of the longest path from s_0 to v.

For the source $v = s_0$, the claim follows directly from the fact that the degenerate path $P = s_0$ with no arcs has weight 1.

For a node $v \neq s_0$, let $u_1, \dots, u_d$ be the predecessor nodes of v, with $d = d^-(v)$. Then

$$\begin{aligned}\sum_{\text{paths } P \text{ from } s_0 \text{ to } v} e(P) &= \sum_{i=1}^{d} \left(\sum_{\text{paths } P' \text{ from } s_0 \text{ to } u_i} e(P')\overleftarrow{q}_{u_iv} \right) \\ &= \sum_{i=1}^{d} \left(\sum_{\text{paths } P' \text{ from } s_0 \text{ to } u_i} e(P') \right) \overleftarrow{q}_{u_iv} = \sum_{i=1}^{d} \overleftarrow{q}_{u_iv} = 1,\end{aligned}$$

since each parenthesizes expression in the last line is 1, by the induction hypothesis, and the backward proportions sum to 1, by assumption (2). ◀

2 Experiments

To test the accuracy of the estimation, we tried the algorithm on two families of DAGs whose size we know. The PYTHON programs by which these preliminary experiments were conducted are available in a public repository[1], together with tables of the results of more extensive experiments.

In our experiments, we have so far investigated only the basic version of the algorithm for node counting, and we have worked exclusively with uniform costs $c(v) = 1$.

Some more interesting potential applications of the method are mentioned in Section 6.3.

[1] https://github.com/guenterrote/estimate-DAG

2.1 Permutations

The nodes of the *permutation DAG* are the $n!$ permutations of the elements $\{1, 2, \ldots, n\}$. The source node is the decreasing permutation $n, n-1, \ldots, 3, 2, 1$, and the successors of a permutation are the permutations that can be reached by swapping two adjacent elements that are out of order, as in bubblesort. The sink is the sorted permutation $123\ldots n$. Figure 2 shows a random path from the source 54321 to the sink 12345 for $n = 5$, leading to the estimate $X \approx 25.6296$. (The true size is $5! = 120$.)

54321	$d_+ = 4,\ d_- = 0,\ W = 1,\ X = 1$
⋀	
54231	$d_+ = 3,\ d_- = 1,\ W = 4,\ X = 5$
⋀	
45231	$d_+ = 2,\ d_- = 2,\ W = 6,\ X = 11$
⋀	
45213	$d_+ = 2,\ d_- = 2,\ W = 6,\ X = 17$
⋀	
45123	$d_+ = 1,\ d_- = 3,\ W = 4,\ X = 21$
⋀	
41523	$d_+ = 2,\ d_- = 2,\ W = 2,\ X = 23$
⋀	
14523	$d_+ = 1,\ d_- = 3,\ W = 1.3333,\ X = 24.3333$
⋀	
14253	$d_+ = 2,\ d_- = 2,\ W = 0.6667,\ X = 25$
⋀	
12453	$d_+ = 1,\ d_- = 3,\ W = 0.4444,\ X = 25.4444$
⋀	
12435	$d_+ = 1,\ d_- = 3,\ W = 0.1481,\ X = 25.5926$
⋀	
12345	$d_+ = 0,\ d_- = 4,\ W = 0.0370,\ X = 25.6296$

Figure 2 A path in the permutation DAG

We have tested the algorithm for the permutation DAG for $n = 8$ and $n = 9$. Table 1 displays the average of up to $m = 10^7$ repetitions of Algorithm P. The true value $n!$ is displayed in the last row. We see that the values gradually converge to an estimate that is in the right ballpark.

We also show the empirical standard deviation (the square root of the empirical variance) of the first m samples, as well as value of the largest sample. The last row gives the precise theoretical value of the standard deviation of the random variable X, computed by the method of Section 3.

We see that the standard deviation is rather high, at least one order of magnitude higher than the mean. We also see that the largest estimate is very much larger than the mean. After $m = 1000$ estimates for $n = 8$, we have seen an estimate of around 4.83×10^7, leading to the outlier mean of 77,136. These phenomena indicate that the estimate is unreliable. Indeed, in the context of backtracking trees, it has often been observed that, while the estimate produced by Path Sampling is unbiased, it has a high uncertainty, and the information it gives is sometimes useless.

For such situations, Chatterjee and Diaconis [8, Section 2] proposed to look at the quantity $\hat{\chi} = \frac{\max\{X_1,\ldots,X_m\}}{X_1+\cdots+X_m}$, see also [17, Eq. (34), p. 50]. A value of $\hat{\chi}$ close to 1 indicates that the mean is essentially determined by a single sample that dominates the other samples by far. Chatterjee and Diaconis argue that a reasonable policy is to trust the experiments when $\hat{\chi}$ is small. Our tables show the Chatterjee–Diaconis score $\hat{\chi}$ in the last column.

Permutations						
	$n=8$			$n=9$		
m	mean $\pm$ std. dev.	max.	$\hat{\chi}$	mean $\pm$ std. dev.	max.	$\hat{\chi}$
10^1	$39{,}331.8 \pm 59{,}061.9$	1.42×10^5	0.36044	$48{,}168.7 \pm 56{,}348.3$	1.68×10^5	0.34803
10^2	$28{,}772.5 \pm 64{,}855.0$	3.42×10^5	0.11875	$134{,}918.1 \pm 4.87\times10^5$	4.21×10^6	0.31195
10^3	$77{,}136.0 \pm 1.55\times10^6$	4.83×10^7	0.62640	$188{,}542.8 \pm 1.33\times10^6$	2.93×10^7	0.15514
10^4	$44{,}629.3 \pm 6.57\times10^5$	4.83×10^7	0.10826	$221{,}679.5 \pm 2.23\times10^6$	1.49×10^8	0.06720
10^5	$42{,}382.8 \pm 1.13\times10^6$	2.89×10^8	0.06821	$372{,}754.8 \pm 1.75\times10^7$	4.94×10^9	0.13250
10^6	$39{,}837.5 \pm 8.32\times10^5$	3.63×10^8	0.00912	$347{,}645.5 \pm 1.23\times10^7$	4.98×10^9	0.01433
10^7	$40{,}820.6 \pm 1.92\times10^6$	5.16×10^9	0.01265	$351{,}350.7 \pm 2.59\times10^7$	4.64×10^{10}	0.01320
	$40{,}320 \pm 1.642\times10^6$	#visited $= 29$		$362{,}880 \pm 1.614\times10^8$	#visited $= 37$	

Table 1 Algorithm P for the permutation DAG. The number of visited nodes is always $1+\binom{n}{2}$, equaling the number of layers, because all source-sink path have the same length.

2.2 Cubes

We take the graph (i. e., the edge skeleton) of the unit cube in n dimensions and orient its edges by some generic objective function. It turns out that, on these instances, Algorithm P works impeccably! At distance d from the source, all nodes have indegree d and outdegree $n-d$. Hence the accumulated contributions to X are $1+n+\binom{n}{2}+\binom{n}{3}+\cdots+\binom{n}{d}+\cdots=2^n$. We always get the correct answer.

2.3 Klee-Minty cubes

Since the behavior of Algorithm P for standard cubes is so uninteresting, we apply the algorithm to *Klee-Minty cubes* instead. Klee and Minty, in 1972, defined a perturbation of the unit n-cube for which they could show that the simplex algorithm, with certain pivot rules, visits all 2^n vertices and thus takes exponential time [15].

Figure 3 shows a version of this polytope with integral vertices, inscribed in a box with exponentially increasing side lengths rather than a cube. The arcs are oriented in the direction of the last coordinate x_n. This orientation of the arcs agrees with the original Klee-Minty cubes. Our version of the Klee-Minty cube in n dimensions is defined by the following linear program:

Maximize x_n

subject to the following $2n$ constraints:

$$\begin{aligned} 0 \le x_1 &\le 1 \\ x_{i-1} \le x_i &\le 2^i - 1 - x_{i-1}, \quad \text{for } i = 2, \ldots, n \end{aligned} \tag{5}$$

We see that the n-dimensional Klee-Minty "box" has a *floor* facet and a *ceiling* facet, where x_n is at its lower bound $x_n = x_{n-1}$ or at its upper bound $x_n = 2^n - 1 - x_{n-1}$, respectively. Both the floor and the ceiling is an affine image of an $(n-1)$-dimensional Klee-Minty box. The floor has the original orientations, and in the ceiling, all orientations are reversed.

Algorithm P essentially performs the simplex algorithm with the *random-edge* pivot rule. For the expected number of steps of this pivot rule on Klee-Minty cubes, there is an easy upper bound of $O(n^2)$ [13]. A matching lower bound of $\Omega(n^2)$ is known only for a random start vertex [4].

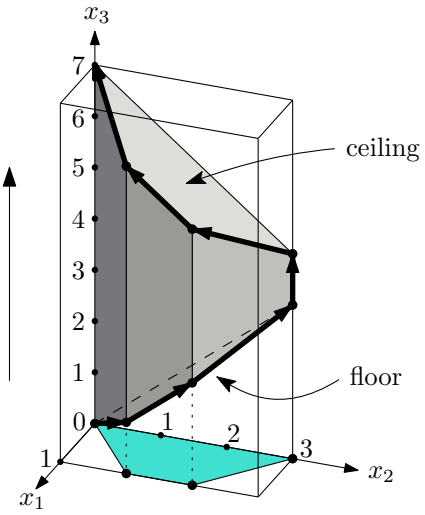

Figure 3 An integral Klee-Minty cube in $n = 3$ dimensions with the directed path through all 2^n vertices. The edges are oriented upwards in the vertical x_3-direction.

Klee–Minty cubes						
	$n = 10$			$n = 12$		
m	mean ± std. dev.	max.	$\hat{\chi}$	mean ± std. dev.	max.	$\hat{\chi}$
10^1	$882.1 \pm 2{,}154.3$	6.96×10^3	0.78904	$1{,}046.4 \pm 2{,}134.4$	6.81×10^3	0.65084
10^2	181.2 ± 717.4	6.96×10^3	0.38409	$478.4 \pm 1{,}484.8$	1.21×10^4	0.25341
10^3	$961.4 \pm 11{,}280.5$	2.85×10^5	0.29669	$1{,}787.6 \pm 24{,}370.0$	6.97×10^5	0.38967
10^4	$711.4 \pm 18{,}195.4$	1.69×10^6	0.23742	$1{,}118.3 \pm 16{,}573.2$	1.05×10^6	0.09420
10^5	$1{,}146.6 \pm 1.42\times10^5$	4.42×10^7	0.38589	$1{,}499.9 \pm 36{,}213.5$	5.76×10^6	0.03842
10^6	$854.8 \pm 65{,}844.1$	4.42×10^7	0.05176	$2{,}649.3 \pm 3.50\times10^5$	2.71×10^8	0.10227
10^7	$866.6 \pm 85{,}253.1$	1.02×10^8	0.01178	$3{,}542.4 \pm 2.27\times10^6$	6.58×10^9	0.18583
	$1{,}024 \pm 3.246\times10^9$			$4{,}096 \pm 1.034\times10^{19}$		
	#visited: avg = 15.7, max = 78			#visited: avg = 20.6, max = 94		

Table 2 Algorithm P for Klee-Minty cubes

Table 2 shows the results of two runs, with $n = 10$ and $n = 12$ dimensions. In comparison to the permutation DAG, we see that the estimate is less accurate, even though the cubes have much fewer nodes. The theoretical standard deviation is very large (around 10^{19} for $n = 12$), but this high standard deviation is not observed empirically.[2]

Appendix B gives some details about the representation that we used for the vertices of the Klee-Minty cubes.

[2] This extreme clash between the predicted and the observed standard deviation raises the suspicion of an error in the formulas or a programming error. However, for smaller values of n, the empirical standard deviation converges very well to the predicted standard deviation.

3 Computing the variance of X

For test DAGs that are small enough so that one can iterate through all nodes, it is possible to compute the variance of the random estimate X for Algorithm P exactly.

We use the following basic result about a mixture of distributions:

▶ **Lemma 6** (Mixture of distributions). *Consider d random variables $Z_1, \dots, Z_d$ with means μ_i and variances var_i. Let Z be the mixture of these variables with proportions $\alpha_1, \dots, \alpha_d$, that is, Z is set to Z_i with probability α_i, where $\sum_{i=1}^{d} \alpha_i = 1$. Then Z has mean*

$$\mu = \sum_{i=1}^{d} \alpha_i \mu_i$$

and variance

$$\mathrm{var} = \sum_{i=1}^{d} \alpha_i \big(\mathrm{var}_i + (\mu_i - \mu)^2\big) = \sum_{i=1}^{d} \alpha_i \mathrm{var}_i + \sum_{1 \le i < j \le d} \alpha_i \alpha_j (\mu_i - \mu_j)^2. \tag{6}$$

These equations can be seen as instances of the *law of total expectation* $E[Z] = E[E[Z|Y]]$ and the *law of total variance* $\mathrm{var}(Z) = E[\mathrm{var}(Z|Y)] + \mathrm{var}(E[Z|Y])$, where $Y \in \{1, \dots, d\}$ is a random variable that determines which Z_i is chosen. The last expression in (6) is an alternative formula, which is sometimes more convenient for hand calculation when d is small.

Let us apply this to Algorithm P. For each node u, we define the random variable $D_u(W)$ to be the value of all future additions to X, assuming that u is entered with weight W. Since the operations on W consist of multiplications by random quantities that are independent of W, it is clear that $D_u(W)$ is a linear function of W, i. e.,

$$D_u(W) = F_u W,$$

for some random factor F_u that depends only on the graph G, together with the forward proportions $\vec{p}_{uv}$ and the arc weights.

We now show how the mean and the variance of the variables F_u can be computed, proceeding from the end of the DAG towards the source, in reverse direction of the arcs. Eventually we arrive at the final estimate $X = F_{s_0}$, and $\mathrm{var}(F_{s_0})$ is the quantity that we are looking for.

Let u be a node with d successors $v_1, \dots, v_d$. Then F_u is $c(u)$ plus a mixture of the d random variables $e_{uv_j}/\vec{p}_{uv_j} \cdot F_{v_j}$, each taken with probability $\alpha_j = \vec{p}_{uv_j}$. Let μ_j and var_j denote the mean and variance of F_{v_j}. Then, by Lemma 6, the mixture has mean

$$\mu_0 = \sum_{j=1}^{d} \vec{p}_{uv_j} \cdot \frac{e_{uv_j}}{\vec{p}_{uv_j}} \cdot \mu_j = \sum_{j=1}^{d} e_{uv_j} \mu_j \tag{7}$$

and variance

$$\begin{aligned} \mathrm{var}(F_u) &= \sum_{j=1}^{d} \vec{p}_{uv_j} \cdot \left[\left(\frac{e_{uv_j}}{\vec{p}_{uv_j}} \right)^2 \cdot \mathrm{var}_i + \left(\frac{e_{uv_j}}{\vec{p}_{uv_j}} \mu_j - \mu_0 \right)^2 \right] \\ &= \sum_{j=1}^{d} \frac{1}{\vec{p}_{uv_j}} \left[e_{uv_j}^2 \mathrm{var}_j + \big(e_{uv_j} \mu_j - \vec{p}_{uv_j} \mu_0 \big)^2 \right] \end{aligned}$$

To get the expectation of F_u, we must not forget to add the cost $c(u)$:

$$E[F_u] = c(u) + \mu_0 = c(u) + \sum_{j=1}^{d} e_{uv_j}\mu_j \tag{8}$$

For a node u with no successors, the formulas specialize to $\mathrm{var}(F_u) = 0$ and $E[F_u] = c(u)$.

For the special case of uniform forward proportions $\vec{p}_{uv_j} = 1/d$, we get the following formulas:

$$\mathrm{var}(F_u) = d\cdot\sum_{j=1}^{d}\left[e_{uv_j}^2\,\mathrm{var}_j + \left(e_{uv_j}\mu_j - \frac{\mu_0}{d}\right)^2\right] = d\cdot\sum_{j=1}^{d} e_{uv_j}^2\,\mathrm{var}_j + \sum_{1\le i<j\le d}\left(e_{uv_i}\mu_i - e_{uv_j}\mu_j\right)^2$$

4 Importance Sampling

How can we use the freedom to pick the successor according to an arbitrary distribution in order to get a better estimate? For a rooted tree, it is known what the ideal forward proportions $\vec{p}_{uv}$ are: They should be proportional to the cost of the subtrees rooted at the children v of node u. If the forward proportions are chosen in this way, the estimate X will always deliver the correct answer.

In practice, one can choose the probability $\vec{p}_{uv}$ of going from u to v based on the properties of v in the graph, e.g., the number of successors or predecessors, or on characteristics of the combinatorial objects that the node v represents. We are entering the realm of heuristic arguments, and the success of any particular rule depends on the type of problem.

We have tried the simple rule to choose the probabilities proportional to the outdegrees of the successor nodes v. (If the outdegree is 0, we have taken 1 instead, to avoid zero probabilities.) Vertices with large outdegree are more likely to reach many nodes. For the permutation DAG, the outdegree (and hence indegree) of a successor v differs by at most ± 1 from u, and hence the effect of the outdegree tends to amplify. This is why it seems to make sense to choose the outdegree as a criterion.

Permutations						
	$n = 8$			$n = 9$		
m	mean $\pm$ std. dev.	max.	$\hat{\chi}$	mean $\pm$ std. dev.	max.	$\hat{\chi}$
10^1	$10{,}564.8 \pm 11{,}944.9$	3.32×10^4	0.31394	$234{,}362.6 \pm 4.41\times10^5$	1.28×10^6	0.54434
10^2	$15{,}677.9 \pm 28{,}749.5$	1.85×10^5	0.11808	$315{,}348.3 \pm 1.03\times10^6$	6.34×10^6	0.20111
10^3	$47{,}912.4 \pm 4.33\times10^5$	1.23×10^7	0.25653	$281{,}586.0 \pm 1.10\times10^6$	1.40×10^7	0.04980
10^4	$39{,}265.0 \pm 2.24\times10^5$	1.23×10^7	0.03130	$312{,}165.1 \pm 3.48\times10^6$	2.36×10^8	0.07569
10^5	$40{,}970.8 \pm 3.82\times10^5$	6.63×10^7	0.01619	$411{,}450.9 \pm 2.05\times10^7$	6.05×10^9	0.14698
10^6	$39{,}777.6 \pm 4.31\times10^5$	2.35×10^8	0.00592	$360{,}495.0 \pm 1.13\times10^7$	6.05×10^9	0.01678
10^7	$40{,}289.8 \pm 5.41\times10^5$	7.25×10^8	0.00180	$372{,}028.6 \pm 4.02\times10^7$	1.01×10^{11}	0.02708
	$40{,}320 \pm 6.316\times10^5$	#visited = 29		$362{,}880 \pm 4.665\times10^7$	#visited = 37	

Table 3 Algorithm P for permutations with Importance Sampling by outdegree

The results are shown in Table 3. We see that the theoretical standard deviation goes down by a factor between 2.6 and 3.5 in comparison with Table 1. In terms of empirical behavior, the comparison is not clear. The experiments are haunted by the occasional occurrences of rare events, such as unusually early appearances of high estimates like 6.05×10^9 and 1.01×10^{11} for $n = 9$.

We tried the same method also for Klee-Minty cubes, with similarly inconclusive results. Appendix A reports these results, as well as some further experiments of only theoretical value, where the size of the reachable node set is taken into account for the forward proportions.

In a different setting, namely the enumeration tree for pseudoline arrangements, it has been observed that the degrees of nodes and their children are strongly correlated, see [20, Appendix E5, Fig. 14], suggesting that importance sampling should be helpful. It indicates that forward proportions chosen proportional to the outdegrees would unterestimate the true size of the subtrees, and thus the influence of the outdegrees on the probabilities should be even stronger than just proportional. However, an experimental evaluation has not been done yet. The nodes in the deeper levels of these enumeration trees have typically several hundred to thousands of successors.

5 Employing a team of agents

5.1 The Team Sampling family of algorithms

Instead of making many independent repetitions of following a single path with Algorithm P, it is better to let several "tokens" or "cooperative agents" move through the graph simultaneously. *From now on, we assume that the arc weights e_{uv} are positive.*

We propose the following general scheme. The algorithm maintains a list A of *active nodes*, and for each element $v \in A$, a positive weight $W[v]$.

Algorithm TS: Team Sampling

1. [Initialize] $A = \{s_0\}$; $W[s_0] := 1$; $X := 0$.

2. While A is nonempty, perform any of the following two steps:

2E. [Expand successors] Choose an arbitrary node u of A, and remove it from A.
Set $X := X + W[u] \cdot c(u)$. // Augment X
For each successor v of u:
If v is not present in A, insert v into A and clear its weight: $W[v] := 0$.
Then, increase its weight to $W[v] := W[v] + W[u]e_{uv}$.

2C. [Condense] Choose an arbitrary subset S of A with at least 2 elements.
Pick a random node u from S, with probabilities proportional to the weights $W[u]$.
Replace S in A by the single element u, and set $W[u]$ to the sum of the weights in S.

3. [Terminate] In the end, return X.

The choice between Step 2E and Step 2C, and the choice of u in Step 2E or of S in Step 2C can be done according to some rule, depending on any information that has been stored or that is currently at hand, or even randomly. The weights can be taken into account. We prove in Section 5.4 that, whatever choices the algorithm makes, it returns an unbiased estimate X of the quantity $c_e(G)$ that was defined in (3) in Theorem 3.

Algorithm P is a special case of this algorithm where Step 2E is always directly followed by Step 2C for the whole set A, i. e., the successors of the current node. In contrast to Path Sampling, Algorithm TS as formulated cannot accommodate a nonuniform selection among the successors. (However, Section 5.5 will introduce nonuniform forward proportions $\vec{p}_{uv}$ through the backdoor.)

It can happen that a node v is removed from A and later reinserted. For example, in the graph of Figure 1, after initially expanding v_1 into the set $A = \{v_2, v_3\}$, the algorithm could first expand v_3, and later v_2, reinserting v_3 into A. This is obviously a waste of effort, and it can be avoided by choosing the nodes in Step 2E in some topological order of the DAG. In

many applications, it is easy to obey this rule. For example, the permutation DAG is graded: All paths from the source to a node v have the same length. More generally, we may have some rank function $r\colon V \to \mathbb{Z}$ or $r\colon V \to \mathbb{R}$ such that $r(v) > r(u)$ for all arcs uv. When we have the graph of a polytope, oriented by some generic linear objective function f, then f can serve as a rank function for the nodes. In other instances, it may not be straightforward to find out if a node v is reachable from a node u. In any case, the correctness of the algorithm does not depend on any proper order in which the nodes are chosen.

Keeping multiple copies of the same node. In the above formulation of Step 2E, we check the presence of a node v in A before trying to insert it, but this is actually not necessary. We can also insert a fresh copy of v into A with a separate weight $W[v]$, (which is initialized to 0 as usual). Accordingly, we have to treat A as a *multiset*. When such a duplication is detected, it is of course expedient to condense multiple copies of the same node v, as a special case of Step 2C; but again, the correctness of the algorithm does not depend on this.

The general scheme of Team Sampling offers a lot of freedom. We describe and test two strategies that specialize this scheme, *Bucket Sampling* and *Crowd Sampling*, both inspired by methods that were proposed for trees in the literature.

5.2 Bucket Sampling (Algorithm BS)

This strategy is modeled after the algorithm of Pang C. Chen [9] from 1992, which was proposed for trees under the name *heuristic sampling*; Knuth [18, Section 7.2.2.9, Algorithm C] called it *Chen subset sampling*. Nodes with similar characteristics are lumped together in a group S that is to be condensed. The idea is that, if the groups are formed in a sensible way, any element $u \in S$ is as good a representative of S as any other element.

More formally, the algorithm uses a *heuristic function* $h\colon V \to U$ from the nodes V to some universe U, which is used to index the *buckets* to which the nodes are assigned. Ideally, the criterion by which nodes are grouped would be that the nodes have similar subtrees (when G is a tree) or similar sets of successors (when G is a DAG). However, these characteristics are usually not available or expensive to compute, so one has to settle for some easier "heuristic" characteristics. In our experiments, we have chosen the outdegree as the heuristic function.

In the original algorithm of Chen [9], a node and its ancestor node cannot be in one bucket, and what we call a bucket is called a *stratum*. This requirement, together with a proper order of processing the strata, ensures that the algorithm visits at most one node per stratum. With our relaxed notion of heuristic function, the stratum terminology is less appropriate, and we have chosen the term bucket instead.

Depending on the character of the universe U, the list of buckets can be organized in an array, a hash table, or a priority queue.

Conceptually, each bucket could store a list containing the (multi-)set of nodes v with the same values $h(v)$. The algorithm repeatedly chooses some nonempty bucket, condenses it, and expands the chosen representative.

However, the following considerations, which were already made for the original heuristic sampling method of [9], show that one only needs to store at most one node per bucket. Suppose we condense a set $S = \{u_1, u_2, u_3\}$, resulting in a representative $u' \in S$, and later condense the set $\{u', u_4, u_5, u_6\}$. It can be easily checked that the total effect is the same as condensing the set $\{u_1, u_2, u_3, u_4, u_5, u_6\}$ in one go. Thus, if we known that u_1, u_2, u_3 will

eventually be condensed (possibly together with more elements that are not yet known), it makes sense to condense them right away, in order to save space.

Therefore, we store at most one node per bucket. In the expansion step, we generate successor nodes and insert them into their buckets. Whenever a node is inserted into a bucket that is already occupied, we immediately condense the set of these two nodes. (This procedure corresponds to the *reservoir sampling* method of selecting a sample from a data stream.)

Experiments. We have tried this approach for the permutation DAG and the Klee-Minty cubes. As a heuristic function, we have taken the outdegree. In each step, we have selected the nonempty bucket with the largest outdegree and expanded the node stored in this bucket. The rationale behind this choice is that nodes closer to the root tend to have larger outdegree. For the permutation DAG, it would have been natural to used the level (rank) as additional information, but we have ignored this for simplicity.

The results are shown in Tables 4 and 5. The number of iterations (reported as "#visited") is larger than for Path Sampling, and in addition, each visit incurs more work than for Path Sampling; therefore we went only up to $m = 10^6$ repetitions. One can see that Bucket Sampling with our very simple bucketing strategy improves the estimate. The standard deviation is reduced to a reasonable scale, in particular for the smaller instances.

We also report as "#generated" the total number of successor nodes that are generated and inserted into buckets; this quantity is representative of the total work of the algorithm. Our digraphs are regular graphs, of degree $n-1$ for the permutation DAGs, and of degree n for the Klee-Minty cubes. On average, half of the incident edges are outgoing edges. Thus the number of generated nodes should be roughly the number of visited nodes times $(n-1)/2$ or $n/2$, respectively. The actual number of generated nodes ("#generated") is just slightly smaller than what one would expect from this calculation.

Permutations						
	$n = 8$			$n = 9$		
m	mean $\pm$ std. dev.	max.	$\hat{\chi}$	mean $\pm$ std. dev.	max.	$\hat{\chi}$
10^1	$34{,}023.3 \pm 41{,}991.4$	1.48×10^5	0.43367	$186{,}271.5 \pm 1.09\times10^5$	3.38×10^5	0.18146
10^2	$47{,}540.9 \pm 69{,}429.7$	4.77×10^5	0.10031	$277{,}473.9 \pm 4.23\times10^5$	3.41×10^6	0.12283
10^3	$39{,}992.2 \pm 47{,}453.0$	4.77×10^5	0.01192	$333{,}151.4 \pm 5.89\times10^5$	7.63×10^6	0.02289
10^4	$39{,}758.2 \pm 54{,}078.2$	1.14×10^6	0.00286	$362{,}520.5 \pm 1.09\times10^6$	7.77×10^7	0.02143
10^5	$40{,}183.6 \pm 59{,}067.6$	2.87×10^6	0.00071	$361{,}473.8 \pm 1.01\times10^6$	1.17×10^8	0.00324
10^6	$40{,}336.0 \pm 61{,}807.6$	5.89×10^6	0.00015	$363{,}775.1 \pm 9.37\times10^5$	1.71×10^8	0.00047
	40,320			362,880		
	#visited: avg = 74.3, max = 95			#visited: avg = 109.6, max = 144		
	#generated: avg = 247.1			#generated: avg = 414.9		

Table 4 Bucket Sampling for the permutation DAG.

5.3 Crowd Sampling (Algorithm CS)

This strategy is inspired by the *stochastic enumeration* (SE) algorithm, which was proposed in 2013 by Reuven Y. Rubinstein [21] to count the number of nodes of a tree. The algorithm has a budget parameter B, imposing a limit on the number of nodes that we want to keep

Klee–Minty cubes						
	$n = 10$			$n = 12$		
m	mean ± std. dev.	max.	$\hat{\chi}$	mean ± std. dev.	max.	$\hat{\chi}$
10^1	1,183.2 ± 1,423.3	4.64×10^3	0.39191	2,930.8 ± 2,237.0	7.84×10^3	0.26759
10^2	1,171.8 ± 2,521.9	1.91×10^4	0.16258	3,486.7 ± 5,502.1	3.70×10^4	0.10600
10^3	1,111.9 ± 3,284.9	7.72×10^4	0.06940	3,711.8 ± 13,201.9	2.79×10^5	0.07510
10^4	1,022.8 ± 3,526.0	2.42×10^5	0.02371	3,964.3 ± 18,223.7	1.15×10^6	0.02899
10^5	1,014.0 ± 3,282.6	3.22×10^5	0.00318	3,993.5 ± 27,014.1	4.98×10^6	0.01246
10^6	1,031.1 ± 4,959.5	2.97×10^6	0.00288	4,104.4 ± 44,370.2	2.05×10^7	0.00500
	1,024 #visited: avg = 193.2, max = 1328 #generated: avg = 936.6			4,096 #visited: avg = 543.1, max = 4889 #generated: avg = 3096.1		

Table 5 Bucket Sampling for Klee-Minty cubes.

for further expansion. The algorithm consists of an alternating sequence of expansion and condensation phases. The main loop is as follows.

CS-2E [Expansion]
Expand every node of the current set A.
Let A' denote the resulting multi-set (or set, after identifying duplicates).
CS-2C [Condensation]
If $|A'| \le B$, let $A := A'$.
Otherwise, randomly partition A' into B sets $S_1 \cup \cdots \cup S_B$ of size $\lfloor \frac{|A'|}{B} \rfloor$ or $\lceil \frac{|A'|}{B} \rceil$. Condense each set S_i to a single element, and let A be the set of these B elements.

We call this method *Crowd Sampling*, because the samples are treated more like an unorganized crowd than a cooperating team.

Tables 6 and 7 show the results of this method. We also report the number of nodes that were saved by condensing multiple copies of the same node after the expansion step ("#saved"). Compared to our Bucket Sampling experiments (Tables 4–5), we see that the standard deviation is further reduced, at the expense of a larger number of visited nodes. The number of visited nodes can be controlled by adjusting the parameter B.

The purpose of Step CS-2C is to reduce the size of the active set A' to B. The method we have proposed is a straightforward way to achieve this, but one can think of many methods that are more sophisticated, taking into account the weights, for example.

5.4 Correctness of the Team Sampling scheme

We use the notation $\mathcal{P}(v \to w)$ for the sum of all path weights from v to w:

$$\mathcal{P}(v \to w) = \sum_{\text{paths } P \text{ from } v \text{ to } w} e(P)$$

▶ **Lemma 7.** *Suppose that at some stage in Algorithm TS, we have a current estimate X^{current} and a current active set A with weights $W[v]$ for $v \in A$. If we perform only expansion steps (Step 2E) until the end of the algorithm, then the value of the estimate X at termination can be predicted deterministically, and it is*

$$X^{\text{current}} + \sum_{v \in A} W[v] f_v, \tag{9}$$

Permutations						
	$n = 8$			$n = 9$		
m	mean ± std. dev.	max.	$\hat{\chi}$	mean ± std. dev.	max.	$\hat{\chi}$
10^1	37,629.0 ± 21,347.7	7.31×10^4	0.19427	335,864.8 ± 5.66×10^5	1.93×10^6	0.57459
10^2	41,642.1 ± 31,007.1	1.77×10^5	0.04256	463,857.8 ± 7.90×10^5	6.76×10^6	0.14564
10^3	40,783.3 ± 34,004.7	2.51×10^5	0.00616	385,896.2 ± 7.23×10^5	1.26×10^7	0.03273
10^4	40,390.6 ± 36,143.8	7.15×10^5	0.00177	365,727.5 ± 6.22×10^5	2.78×10^7	0.00760
10^5	40,330.1 ± 34,894.1	8.50×10^5	0.00021	366,894.1 ± 5.96×10^5	2.89×10^7	0.00079
10^6	40,336.7 ± 35,324.7	1.18×10^6	0.00003	363,448.7 ± 5.95×10^5	6.57×10^7	0.00018
	40,320			362,880		
	#visited: avg = 260.3, max = 266			#visited: avg = 341.1, max = 348		
	#generated: avg = 893.2			#generated: avg = 1329.5		
	#saved: avg = 189.6 = 21.2%			#saved: avg = 256.0 = 19.3%		

Table 6 Crowd Sampling for the permutation DAG with a budget $B = 10$.

Klee–Minty cubes						
	$n = 10$			$n = 12$		
m	mean ± std. dev.	max.	$\hat{\chi}$	mean ± std. dev.	max.	$\hat{\chi}$
10^1	1,508.5 ± 1,520.0	4.91×10^3	0.32530	2,741.1 ± 2,785.5	9.82×10^3	0.35839
10^2	939.6 ± 998.9	5.31×10^3	0.05647	4,860.2 ± 14,275.1	1.38×10^5	0.28425
10^3	1,028.3 ± 1,292.8	1.62×10^4	0.01578	3,988.3 ± 8,651.3	1.38×10^5	0.03464
10^4	1,001.7 ± 1,342.2	4.10×10^4	0.00409	4,124.8 ± 11,867.5	6.98×10^5	0.01692
10^5	1,018.4 ± 1,442.2	6.95×10^4	0.00068	4,069.6 ± 12,271.1	1.07×10^6	0.00263
10^6	1,024.6 ± 1,483.7	1.28×10^5	0.00012	4,082.1 ± 13,309.8	3.93×10^6	0.00096
	1,024			4,096		
	#visited: avg = 709.3, max = 1441			#visited: avg = 937.7, max = 1839		
	#generated: avg = 2486.1			#generated: avg = 3892.1		
	#saved: avg = 392.2 = 15.8%			#saved: avg = 416.2 = 10.7%		

Table 7 Crowd Sampling for Klee-Minty cubes with a budget $B = 10$.

for some factors f_v that depend only on the graph G, the costs $c(v)$, and the arc weights e_{vw}:

$$f_v = \sum_{w \in W} \mathcal{P}(v \to w) c(w) \tag{10}$$

In particular, an expansion step does not change the value of (9).

Proof. It is clear that the last sentence implies the whole lemma, since, at termination, $A = \emptyset$, and X^{current} is the final value of X.

Let $v_1, \ldots, v_d$ be the $d = d^+(u)$ successors of u. Then f_u satisfies the relation

$$f_u = c(u) + \sum_{i=1}^{d} e_{uv_i} f_{v_i}. \tag{11}$$

Expanding vertex $u \in A$ adds $c(u)W[u]$ to X^{current}, and it adds $e_{uv_i}W[u]$ to each $W[v_i]$, so the expression (9) increases by $W[u](c(u) + \sum_{i=1}^{d} e_{uv_i} f_{v_i})$. On the other hand, u is removed from A, causing a decrease by $W[u]f_u$. By (11) these two changes cancel out. ◀

▶ **Lemma 8.** *After a condensation step, the expected value of* $\sum_{v\in A} W[v]f_v$ *is the same as before.*

Proof. Condensing replaces a set S by one of its members $u \in S$, where u is chosen with probability $W[u]/\sum_{v\in S} W[v]$. Let $A' = A \setminus S \cup \{u\}$ be the new active set, and let W' denote the new weights. The new weight of u is $W'[u] = \sum_{v\in S} W[v]$. The weights in $A \setminus S$ are unchanged. Thus we get

$$E\left[\sum_{v\in A'} W'[v]f_v\right] = \sum_{v\in A\setminus S} W[v]f_v + \sum_{u\in S}\Big(\Pr[u \text{ is chosen}] \times W'[u]f_u\Big) \tag{12}$$

$$= \sum_{v\in A\setminus S} W[v]f_v + \sum_{u\in S}\left(\frac{W[u]}{\sum\limits_{v\in S} W[v]} \times \sum_{v\in S} W[v]f_u\right) \tag{13}$$

$$= \sum_{v\in A\setminus S} W[v]f_v + \sum_{u\in S} W[u]f_u = \sum_{v\in A} W[v]f_v, \tag{14}$$

as claimed. ◀

▶ **Theorem 9.** *Algorithm TS produces an estimate with the same expectation as Algorithm P:*

$$E[X] = c_e(G) = \sum_{v\in V} \mathcal{P}(s_0 \to v)c(v)$$

Proof. Lemmas 7 and 8 together imply that, at any time during the algorithm,

$$E[X^{\text{final}}] = X^{\text{current}} + \sum_{v\in A} W[v]f_v.$$

If we apply this to the starting situation of Algorithm TS, with $A = \{s_0\}$, $W[s_0] = 1$, and $X^{\text{current}} = 0$, we get $E[X^{\text{final}}] = f_{s_0}$, which equals the claimed quantity. ◀

We mention that this analysis gives a combinatorial interpretation of the expectation of the random variables F_v introduced in Section 3 to compute the variance of the estimate X. Comparing (11) with (8) and taking note of the fact that μ_j in (8) stands for $E[F_{u_j}]$, we see that f_v and $E[F_v]$ satisfy the same recurrence relation. If v is a sink, (11) and (8) specify the same values $f_v = E[F_v] = c(v)$, which can be used as initial conditions. Thus:

▶ **Proposition 10.** *The expected value of* F_v *equals* f_v. ◀

5.5 How to select a representative node in the condensation step

As mentioned, Crowd Sampling was modeled after the Stochastic Enumeration algorithm of Rubinstein [21], which works for trees. For a tree, the algorithm becomes very simple: In the expansion step, all backward proportions $\overleftarrow{q}_{uv}$ are 1, and hence the children just get the same weights as their parents. If the resulting set A' of children is too big, it is simply replaced by a random sample of size B, and the weight is accordingly multiplied by $|A'|/B$. All nodes have the same weight.

Our Crowd Sampling method mimics the behavior of this algorithm only when $|A'|$ is a multiple of B. Otherwise, Crowd Sampling generates nodes of different weight, introducing irregularities into the process. In fact, for DAGs, nodes of different weight necessarily appear. For a set with weights, the notion of "a random sample of size B" is not obvious; see the survey [12] for two different potential interpretations. Let us take an engineering approach

and formulate what is required to ensure that the estimate remains unbiased. It will turn out that this actually leaves a lot of freedom in performing the condensation step.

Let the set S contain k nodes with weights $W_1, W_2, \ldots, W_k$. Suppose we decide that Step 2C should randomly select the i-th node with probability p_i, and assign weight W_i' to it in case it is selected. If we look at (12–14) in the proof of Lemma 8, we see that what we need for unbiasedness is the equation

$$p_1 W_1' f_1 + \cdots + p_k W_k' f_k = W_1 f_1 + \cdots + W_k f_k. \tag{15}$$

Since we don't know the values f_i, this equation should hold for all $f_1, \ldots, f_k$. Comparing coefficients leads to k equations

$$p_i W_i' = W_i, \tag{16}$$

for $i = 1, \ldots, k$. Together with the equation $\sum_{i=1}^{k} p_i = 1$, this gives $k+1$ equations for $2k$ unknowns p_i, W_i', which is by far not enough to define a unique solution.[3]

The condensation step of Algorithm TS chooses $W_i' = W := W_1 + \cdots + W_k$ and $p_i = W_i/W$, but it is not clear why this choice should be preferable over other choices. In fact, we can select arbitrary probabilities p_i and set $W_i' := W_i/p_i$. In this way, we can reinstate the forward proportions of Algorithm P, which Algorithm TS in its original form could not handle.

5.6 The variance for Team Sampling strategies

For particular strategies that specialize the Team Sampling procedure, it should in principle be possible to compute the exact variance of the estimate X, if the whole graph G with all forward and backward proportions is given. The general idea is that each condensation step contributes to the variance, because it replaces a set S by a single sample.

Knuth [18, Section 7.2.2.9, Theorem V] has given a method to compute the variance for Chen's version [9] of Bucket Sampling, where the heuristic function partitions the nodes into strata in an acyclic way. It would be interesting to get a similar analysis for Crowd Sampling.

6 Conclusion

We have proposed two very general methods for getting unbiased estimates for the size of a DAG. There is ample opportunity to come up with more ideas and try them out; we have barely scratched the surface with our experiments. For example, one can combine the ideas of Bucket Sampling and Crowd Sampling, allowing up to B nodes to occupy one bucket.

For the choice of forward proportions $\vec{p}_{uv}$, Importance Sampling gives some guidelines about what one wants to achieve (see Section 4). By contrast, the role of the backward proportions $\overleftarrow{q}_{uv}$ is not understood at all. One may even allow them to become zero: $\overleftarrow{q}_{uv} = 0$ effectively means that the sample is ignored when the algorithm traverses the arc uv.

6.1 Provable approximation quality?

Vaisman and Kroese [24, Theorem 4.3] showed that Stochastic Enumeration is a fully polynomial randomized approximation scheme (FPRAS) for counting the number of nodes

[3] We may even permit $\sum p_i < 1$, effectively throwing away the sample with probability $1 - \sum p_i$. In the single path variation (Algorithm P), the algorithm gets a chance to finish early and take a rest. This comes, of course, at the expense of a larger fluctuation of the estimate.

of trees, for *most* trees in a certain (rather restricted) class of random trees.[4] It would be interesting to see whether Path Sampling or Team Sampling leads to an FPRAS for some combinatorial counting problems. We mention a few such problems below.

6.2 Extension to arc weights

The extension to estimating the sum of all arc weight in an arc-weighted graph is straightforward: When proceeding from u to v, the algorithm adds $W \cdot d^+(u) \cdot c(uv)$ to X, where $c(uv)$ is some cost function associated to the arc uv. Conceptually, we could simply subdivide every arc and let the newly inserted nodes take over the role of the arcs.

6.3 Some applications

6.3.1 Vertices of a polytope

For a polytope, a linear objective function induces an orientation of the graph of the polytope: It directs each edge in the direction of improvement of the objective function, provided that the objective function is sufficiently generic and does not assign the same value to adjacent vertices.

Avis and Devroye have used the tree version of Algorithm P to estimate the number of vertices of a polytope [3]. The simplex algorithm generates a path from each vertex of a polytope to the optimum vertex, and thereby defines a spanning tree of the vertices. This spanning tree is the core of the *reverse search* method for enumerating all vertices, and it is the tree to which Avis and Devroye apply Algorithm P. Salomone, Vaisman, and Kroese [22] have applied the Stochastic Enumeration method (or Crowd Sampling) to this tree.

However, the graph of a polytope whose edges are oriented by an objective function is naturally a DAG. We intend to compare our approach against the tree-based estimation methods of [3] and [22].[5][6]

6.3.2 Polyominoes

Various type of polyominoes have been counted by the *transfer-matrix* method, which is a dynamic-programming algorithm for counting that avoids to enumerate the polyominoes individually, see [10] for an overview and [5, 6] for recent advances. As is the case for many dynamic-programming algorithms, this can be interpreted as accumulating (possibly weighted) source-sink paths in a DAG. These algorithms are run to get exact counts, exhausting limits of time and memory consumption. With our approach, the same ideas can be leveraged to get estimates far beyond the polyomino sizes for which exact computation is feasible. This

[4] In the statement of this theorem, the budget parameter B for the algorithm is not specified. It must be set to kB', where B' is defined in [24, Theorem 4.1, p. 52], depending polynomially on the tree height and the maximum degree k, as well as the parameters of the random tree model (Radislav Vaisman, personal communication, July 2025).

[5] Our discussion ignores an important technicality: The objects that the simplex algorithm visits are actually the *feasible bases*, and for polytopes that are not simple, a vertex can correspond to several bases. The standard way to treat such *degenerate* or nonsimple polytopes, where more than n facets meet in a vertex, is to perturb the constraints, resulting in a simple polytope, and count only one of the many vertices that correspond to a vertex of the original polytope. See [3] for details.

[6] We mention that the permutation DAG is also obtainable as the graph of a polytope, namely the *permutahedron*: The vertex coordinates of the permutahedron are given by the inverse permutations of the permutations that form the nodes of the permutation DAG as described here. For any generic objective function, the directed graph of the permutahedron edges is isomorphic to the permutation DAG.

might be an alternative to Markov-chain based sampling techniques [23]. With appropriate weights $c(v)$, one can also gather statistics about various properties of polyominoes.

In fact, the essence of the dynamic-programming method in this case can be regarded as condensing the enumeration tree into a DAG by merging nodes that have the same subtrees. (In this spirit, Conway's reference to the DAG in [10, Fig. 2] as a "tree" is perhaps not completely off-track.) Bucket sampling is a relaxation of this approach: We are allowed to merge nodes whose subtrees are different (but hopefully at least similar).

6.3.3 Triangulations of a planar point set

Alvarez and Seidel [2] have given an algorithm with runtime $O(2^n n^2)$ for counting the triangulations of an n-point set in the plane. They model triangulations as source-sink paths in a DAG with $n2^{n-3}$ nodes. Our algorithms can therefore be readily applied. The quality of the resulting approximation remains to be explored. The state of the art about approximate counting of triangulations is rather weak: There is only a subexponential-time approximate counting method with a subexponential multiplicative error [1].

6.3.4 Linear extensions of a two-dimensional partial order

If we start from an arbitrary permutation $\pi = (\pi_1, \ldots, \pi_n)$ in the permutation DAG, the reachable nodes are the linear extensions of a two-dimensional partial order $\prec$, which is defined by the rule $i \prec j :\Leftrightarrow i < j \wedge \pi_i \le \pi_j$. Thus our path sampling methods can be used to estimate the number of linear extensions of such a partial order. The computation of indegrees must be adjusted to count only those predecessor permutations that can be reached from π.

Counting the linear extensions is #P-complete, even for a two-dimensional partial order, as proved by Dittmer and Pak [11, Theorem 1.3].[7] In any case, there already exists a fully polynomial randomized approximation scheme for the number of linear extensions of an arbitrary poset, because the number of linear extensions is directly related to the volume of the order polytope, and the polytope volume can be approximated by Markov chain Monte Carlo methods, see for example [7, Section 3] for details and references. Our path sampling methods might lead to alternative approximation schemes that are simpler.

Acknowledgment. The ideas of this paper were inspired by discussions initiated at the workshop on *Computational Polyhedral Geometry*[8], which took place on June 26, 2025 as part of the Computational Geometry Week (CG Week) in Kanazawa, Japan. I thank Laura Casabella and Dante Luber for the organizing this workshop, and the other participants for the lively discussions.

References

1 Victor Alvarez, Karl Bringmann, Saurabh Ray, and Raimund Seidel. Counting triangulations and other crossing-free structures approximately. *Computational Geometry*, 48(5):386–397, 2015. Special Issue on the 25th Canadian Conference on Computational Geometry (CCCG). doi:10.1016/j.comgeo.2014.12.006.

[7] There is a subsequent journal article by the same authors with the same title, which, however, omits this result and does not even mention it.

[8] https://socg25.github.io/workshops.html

2 Victor Alvarez and Raimund Seidel. A simple aggregative algorithm for counting triangulations of planar point sets and related problems. In *Proceedings of the 29th Annual Symposium on Computational Geometry (SoCG'13)*, pages 1–8, New York, NY, USA, 2013. Association for Computing Machinery. doi:10.1145/2462356.2462392.

3 David Avis and Luc Devroye. Estimating the number of vertices of a polyhedron. *Information Processing Letters*, 73(3):137–143, 2000. doi:10.1016/S0020-0190(00)00011-9.

4 József Balogh and Robin Pemantle. The Klee–Minty random edge chain moves with linear speed. *Random Structures & Algorithms*, 30(4):464–483, 2007. doi:10.1002/rsa.20127.

5 Gill Barequet and Gil Ben-Shachar. Counting polyominoes, revisited. In *Proceedings of the 2024 Symposium on Algorithm Engineering and Experiments (ALENEX)*, pages 133–143. SIAM, 2024. doi:10.1137/1.9781611977929.10.

6 Gill Barequet and Gil Ben-Shachar. Counting polyominoes, revisited. Preprint, 2025. Submitted. doi:10.21203/rs.3.rs-4304962/v1.

7 Graham Brightwell and Peter Winkler. Counting linear extensions. *Order*, 8:225–242, 1991. doi:10.1007/BF00383444.

8 Sourav Chatterjee and Persi Diaconis. The sample size required in importance sampling. *Ann. Appl. Probab.*, 28(2):1099–1135, 2018. doi:10.1214/17-AAP1326.

9 Pang C. Chen. Heuristic sampling: A method for predicting the performance of tree searching programs. *SIAM Journal on Computing*, 21(2):295–315, 1992. doi:10.1137/0221022.

10 Andrew R. Conway. The design of efficient dynamic programming and transfer matrix enumeration algorithms. *Journal of Physics A: Mathematical and Theoretical*, 50(35):353001 (28 pp.), aug 2017. arXiv:1610.09806, doi:10.1088/1751-8121/aa8120.

11 Samuel Dittmer and Igor Pak. Counting linear extensions of restricted posets, 2018. arXiv:1802.06312.

12 Pavlos S. Efraimidis. Weighted random sampling over data streams. In Christos Zaroliagis, Grammati Pantziou, and Spyros Kontogiannis, editors, *Algorithms, Probability, Networks, and Games: Scientific Papers and Essays Dedicated to Paul G. Spirakis on the Occasion of His 60th Birthday*, pages 183–195. Springer International Publishing, Cham, 2015. arXiv:1012.0256, doi:10.1007/978-3-319-24024-4_12.

13 Bernd Gärtner, Martin Henk, and Günter M. Ziegler. Randomized simplex algorithms on Klee-Minty cubes. *Combinatorica*, 18(3):349–372, 1998. doi:10.1007/PL00009827.

14 Marshall Hall and D. E. Knuth. Combinatorial analysis and computers. *The American Mathematical Monthly*, 72(2):21–28, 1965. doi:10.2307/2313307.

15 Victor Klee and G. J. Minty. How good is the simplex algorithm? In O. Shisha, editor, *Inequalities III*, pages 159–175. Academic Press, New York, 1972.

16 Donald E. Knuth. Estimating the efficiency of backtrack programs. *Mathematics of Computation*, 29:122–136, 1975. URL: https://www.ams.org/journals/mcom/1975-29-129/S0025-5718-1975-0373371-6/.

17 Donald E. Knuth. *Combinatorial Algorithms, Part 2*, volume 4B of *The Art of Computer Programming*. Addison-Wesley, 2023.

18 Donald E. Knuth. *Combinatorial Algorithms, Part 3*, volume 4C of *The Art of Computer Programming*. Addison-Wesley, 2027+. In preparation. Draft of Section 7.2.2.9, "Estimating Backtrack Costs" at https://cs.stanford.edu/~knuth/fasc9c.ps.gz, version March 27, 2022.

19 Paul W. Purdom. Tree size by partial backtracking. *SIAM Journal on Computing*, 7(4):481–491, 1978. doi:10.1137/0207038.

20 Günter Rote. NumPSLA – an experimental research tool for pseudoline arrangements and order types. In Jan Kratochvíl and Giuseppe Liotta, editors, *41st European Workshop on Computational Geometry (EuroCG 2025)*, pages 18:1–18:8, April 2025. arXiv:2503.02336.

21 Reuven Rubinstein. Stochastic enumeration method for counting NP-hard problems. *Methodology and Computing in Applied Probability*, 15:249–291, 2013. doi:10.1007/s11009-011-9242-y.

22 Robert Salomone, Radislav Vaisman, and Dirk Kroese. Estimating the number of vertices in convex polytopes. In *Proceedings of the 4th Annual International Conference on Operations Research and Statistics (ORS 2016), Singapore, 2016*, pages 96–105. Global Science and Technology Forum, Singapore, 2016. doi:10.5176/2251-1938_ORS16.25.

23 D. Stauffer. Monte Carlo study of density profile, radius, and perimeter for percolation clusters and lattice animals. *Phys. Rev. Lett.*, 41:1333–1336, Nov 1978. doi:10.1103/PhysRevLett.41.1333.

24 Radislav Vaisman and Dirk P. Kroese. Stochastic enumeration method for counting trees. *Methodology and Computing in Applied Probability*, 19:31–73, 2017. doi:10.1007/s11009-015-9457-4.

A Importance Sampling for Klee-Minty cubes

As mentioned in Section 4, we have tried Importance Sampling by outdegree also for Klee-Minty cubes. In comparison to Table 2, the results of Table 8 show a substantial improvement in the theoretical standard deviation, by several orders of magnitude. Like for permutations, the comparison in terms empirical behavior is not at all clear. Even with 10 million samples, the fluctuations are too high to make a reasonable assessment.

Klee–Minty cubes						
	$n = 10$			$n = 12$		
m	mean ± std. dev.	max.	$\hat{\chi}$	mean ± std. dev.	max.	$\hat{\chi}$
10^1	87.5 ± 102.1	3.34×10^2	0.38153	$2{,}028.6 \pm 3{,}386.0$	1.02×10^4	0.50207
10^2	358.4 ± 995.7	7.46×10^3	0.20824	$2{,}533.0 \pm 13{,}646.2$	1.33×10^5	0.52576
10^3	$649.8 \pm 3{,}521.7$	9.34×10^4	0.14378	$1{,}699.2 \pm 9{,}894.0$	1.79×10^5	0.10549
10^4	$904.4 \pm 24{,}382.4$	2.38×10^6	0.26310	$2{,}084.0 \pm 30{,}840.8$	2.62×10^6	0.12584
10^5	$863.3 \pm 15{,}913.2$	2.91×10^6	0.03368	$2{,}686.7 \pm 62{,}038.0$	1.05×10^7	0.03895
10^6	$963.2 \pm 33{,}681.1$	1.74×10^7	0.01808	$4{,}501.2 \pm 8.53\times10^5$	6.49×10^8	0.14407
10^7	$986.2 \pm 58{,}292.5$	9.65×10^7	0.00979	$4{,}392.0 \pm 1.20\times10^6$	3.00×10^9	0.06841
	$1{,}024 \quad \pm 2.141\times10^6$			$4{,}096 \quad \pm 7.762\times10^{10}$		
	#visited: avg = 22.4, max = 80			#visited: avg = 29.2, max = 104		

Table 8 Algorithm P for Klee-Minty cubes with Importance Sampling by outdegree

A.1 Importance Sampling by the size of the reachable set

As mentioned, when Path Sampling is applied to a rooted tree, the ideal forward proportions are known: The probabilities $\vec{p}_{uv}$ should be proportional to the cost of the subtrees rooted at the children v of node u. For DAGs, it is not clear whether such a perfect choice of forward proportions that completely eliminates variations of X exists. As a substitute for the size of the subtree rooted at v, we may take the size R_v of the node set reachable from v.

For Klee-Minty cubes, we are in the lucky situation that the the size R_v of the reachable set is readily available: The Klee-Minty cube of dimension n has a directed Hamilton path, in which the last coordinate x_n increases in an arithmetic progression $0, 1, 2, \ldots, 2^n - 1$, see Figure 3. Thus, $R_v = 2^n - x_n(v)$.

We made two experiments where R_v is taken into account for computing the forward proportions. We emphasize that this is not suitable in practical applications: If the size of the reachable set were available, there would be no need to compute an estimate by running

Algorithm P. However, the experiment teaches us what we might be looking for in the choice of forward proportions.

In the first experiment (Table 9), we simply chose the probabilities $\vec{p}_{uv}$ for the successors v of a node u proportional to R_v. We see no improvement over simply using the outdegree.

In the second experiment (Table 10), we took into account the fact that a node v with large indegree is visited from several predecessors. We therefore chose the probabilities proportional to $R(v)/d^-(v)$. Here we get much better estimates, and we see a substantial reduction of the standard deviation. While these weights are not perfect, they indicate the direction in which one should look. Conceivably, a perfect choice of parameters can be obtained by adjusting also the backward proportions $\overleftarrow{q}_{uv}$.

Klee–Minty cubes						
	$n = 10$			$n = 12$		
m	mean ± std. dev.	max.	$\hat{\chi}$	mean ± std. dev.	max.	$\hat{\chi}$
10^1	202.2 ± 217.4	6.97×10^2	0.34488	994.4 ± 1,564.7	4.56×10^3	0.45878
10^2	473.9 ± 1,054.7	9.21×10^3	0.19432	1,171.3 ± 3,682.9	3.44×10^4	0.29384
10^3	976.3 ± 10,248.4	2.90×10^5	0.29717	2,154.3 ± 16,630.3	4.04×10^5	0.18755
10^4	1,121.8 ± 30,028.1	2.82×10^6	0.25136	2,337.7 ± 30,091.7	2.26×10^6	0.09668
10^5	897.2 ± 13,521.5	2.82×10^6	0.03143	3,107.8 ± 98,543.9	2.37×10^7	0.07613
10^6	1,026.2 ± 61,065.3	4.95×10^7	0.04828	3,255.6 ± 1.65×10^5	7.04×10^7	0.02163
10^7	1,006.2 ± 65,644.4	1.29×10^8	0.01282	3,635.2 ± 1.01×10^6	3.11×10^9	0.08555
	1,024 ± 2.080×10^6			4,096 ± 6.025×10^{11}		
	#visited: avg = 22.4, max = 80			#visited: avg = 29.2, max = 112		

Table 9 Algorithm P for Klee-Minty cubes with Importance Sampling by the size of the reachable set

Klee–Minty cubes						
	$n = 10$			$n = 12$		
m	mean ± std. dev.	max.	$\hat{\chi}$	mean ± std. dev.	max.	$\hat{\chi}$
10^1	583.2 ± 611.6	2.07×10^3	0.35412	2,778.3 ± 5,045.8	1.67×10^4	0.60117
10^2	1,299.8 ± 4,881.0	4.77×10^4	0.36690	2,231.1 ± 5,569.1	4.60×10^4	0.20638
10^3	897.7 ± 2,388.1	4.77×10^4	0.05313	4,351.1 ± 55,366.3	1.74×10^6	0.39894
10^4	976.6 ± 3,751.7	1.52×10^5	0.01551	3,611.7 ± 46,923.6	4.00×10^6	0.11076
10^5	1,066.5 ± 19,675.0	5.92×10^6	0.05552	3,851.3 ± 58,920.5	1.00×10^7	0.02597
10^6	1,034.9 ± 12,622.9	6.65×10^6	0.00643	4,031.7 ± 1.21×10^5	7.25×10^7	0.01799
10^7	1,023.2 ± 8,554.4	9.69×10^6	0.00095	4,043.8 ± 1.38×10^5	2.93×10^8	0.00725
	1,024 ± 1.071×10^4			4,096 ± 9.419×10^5		
	#visited: avg = 42.5, max = 112			#visited: avg = 58.7, max = 160		

Table 10 Algorithm P for Klee-Minty cubes with Importance Sampling by the size of the reachable set divided by the indegree

B Details about implementing Klee-Minty cubes

The natural way to associate a bitvector $(y_1, \ldots, y_n)$ to the vertices of a Klee-Minty cube (5) (Section 2.3) is derived from the standard cube: $y_i = 1$ if x_i is at the ceiling, i. e.,

$x_i = 2^i - 1 - x_{i-1}$, and $y_i = 0$ if x_i is at the floor, i. e., $x_i = x_{i-1}$. The n neighbors of $(y_1, \ldots, y_n)$ are obtained by simply flipping a single bit. The downside of this representation is that it is not easy to see whether an edge is improving.

Our program uses a different representation $(z_1, \ldots, z_n)$, which was introduced by Gärtner, Henk, and Ziegler [13]:

$$\begin{aligned} z_n &= y_n \\ z_{n-1} &= y_n \oplus y_{n-1} \\ z_{n-2} &= y_n \oplus y_{n-1} \oplus y_{n-2} \\ &\vdots \\ z_1 &= y_n \oplus y_{n-1} \oplus y_{n-2} \oplus \cdots \oplus y_2 \oplus y_1 \end{aligned}$$

It is the sequence of partial sums of the y_i's using addition modulo 2, or the exclusive-or operation $\oplus$.

The outgoing arcs are identified by the zero bits in this representation. If we follow such an arc, we must flip the i-th bit together with all lower-order bits. The reason is that while the orientation of all incident edges with higher index is maintained, the lower-index directions are perverted.

More concretely, we represent a vertex by the n-bit integer $(z_n z_{n-1} \ldots z_2 z_1)_2$. It turns out that the value of this integer is equal to the x_n-coordinate of the vertex.

C Purdom's partial backtracking method

Purdom [19], in 1978, suggested the following variation of Knuth's Path Sampling algorithm for trees: Instead of proceeding to a single child, proceed to at most m randomly selected samples among the children. The parameter m can itself be chosen at random. In particular, Purdom looked at the setting of binary trees, and he proposed to choose $m = 2$ or $m = 1$ with probability p and $1 - p$ respectively.

He analyzed analytically the number of visited nodes as well as the variance of the estimated tree size, for a certain class of random binary trees. He also made experiments with a backtracking program.

This method could be easily adapted to DAGs, but it will probably not compete with the more sophisticated Team Sampling methods, which allow a more direct control of the number of visited nodes. Still we want to cite one sentence from the conclusion of the paper: "The analysis suggests that good results can be obtained by choosing the number of branches to investigate so that each level, from the root down to the bulk of the nodes, is examined at about the same frequency." This is in line with the strategy of the Crowd Sampling scheme, which sets a fixed budget B. In case of a tree, Crowd Sampling would indeed look at up to B nodes in each level.